RAPPORT DE TRAVAUX PRATIQUES

Internet des objets — Atelier 6, partie 2

Supervision d'une salle intelligente

Chaîne IoT complète, broker MQTT et Socket.IO

Acquisition DHT22, nœuds ESP32, broker Mosquitto, serveur Node.js et tableau de bord temps réel

Ce rapport décrit une chaîne complète de supervision environnementale d'une salle (« Smart Room »). Deux cartes ESP32, chacune équipée d'un capteur DHT22, mesurent la température et l'humidité et les publient sur un broker MQTT. Un serveur Node.js valide les messages puis les diffuse en temps réel via Socket.IO.

2× ESP32 + DHT22 → MQTT Broker (Mosquitto) → Node.js / Express → Socket.IO → Dashboard

PRÉSENTÉ PAR

Youssef El Ajbari et Monssef Bougaidan

3e année — IDAI

Module : Internet des objets

30 octobre 2026

Année universitaire 2026–2027

ESP32 · DHT22 · MQTT/Mosquitto · WebSocket · Node.js

Sommaire

1 Introduction 1

2 Architecture du système 1

3 Couche capteurs : firmware ESP32 4

4 Couche serveur : Node.js 7

5 Interface : tableau de bord 10

6 Mise en œuvre et tests 11

7 Analyse critique et pistes d'amélioration 12

8 Conclusion 13

1 Introduction

1.1 Contexte et objectifs

La supervision à distance de grandeurs physiques (température, humidité) repose sur une chaîne en trois maillons : des nœuds de mesure, un moyen de transport léger, et une application qui exploite les données. Le projet réalise cette chaîne de bout en bout pour une salle de travaux pratiques, avec des outils courants de l'Internet des objets.

Objectifs du projet

- acquérir la température et l'humidité toutes les 5 secondes sur deux nœuds ESP32 distincts ;
- transporter les mesures avec le protocole MQTT (modèle publication/abonnement) ;
- valider les données côté serveur (types et plages de valeurs) avant toute diffusion ;
- afficher les valeurs en temps réel sur un tableau de bord web et signaler tout dépassement du seuil de 30 °C.

1.2 Périmètre du rapport

Le rapport est construit autour des scripts listés dans le tableau 1. Les trois captures d'écran fournies servent de preuve de fonctionnement (section 6).

Fichier	Rôle	Lignes
esp32/main.c	Firmware du nœud `ESP32_01` (DHT22 sur GPIO 15)	155
esp32/esp2.c	Firmware du nœud `ESP32_02` (DHT22 sur GPIO 17)	155
nodeserver/server.js	Passerelle MQTT → Socket.IO et API REST	145
nodeserver/public/dashboard.html	Tableau de bord (client Socket.IO)	189
nodeserver/package.json	Dépendances et commande de démarrage	–

TABLE 1 – Scripts étudiés.

2 Architecture du système

2.1 Vue d'ensemble

Le système comprend quatre éléments (figure 1). Les deux ESP32 publient leurs mesures sur le broker MQTT par le réseau Wi-Fi. Le serveur Node.js s'abonne au broker, valide les messages puis les pousse vers les navigateurs connectés. Le serveur se connecte au broker sur `127.0.0.1:1883` : il s'exécute donc sur la même machine que le broker.

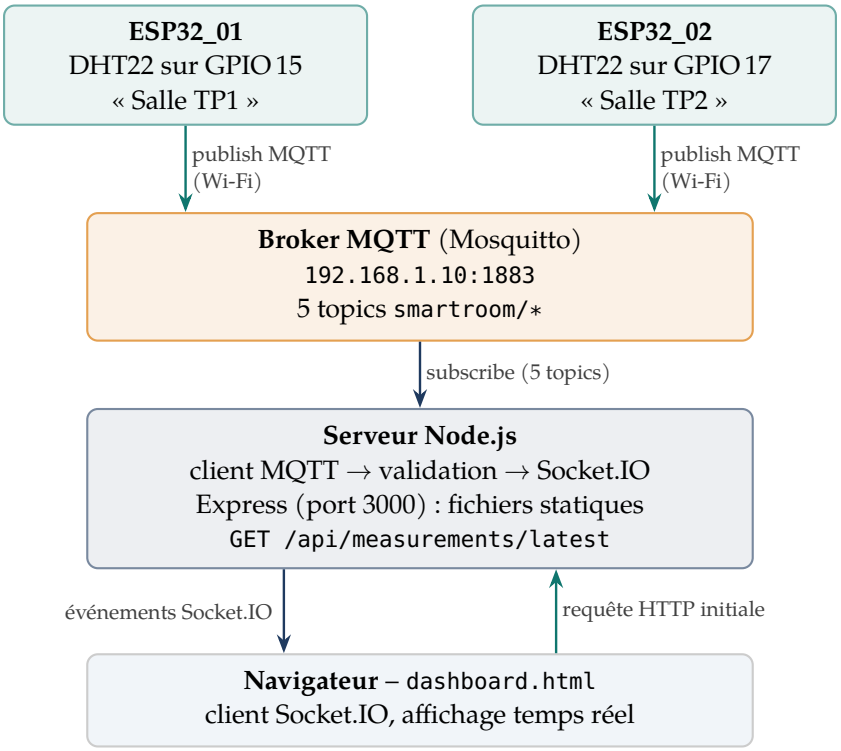


Figure 1 – Architecture générale de la Smart Room.

2.2 Matériel et technologies

Élément	Rôle et remarques
ESP32 (×2)	Microcontrôleur avec Wi-Fi ; exécute le firmware Arduino.
Capteur DHT22	Température et humidité relative ; plage de mesure −40 à 80 °C et 0 à 100 % ; période d'échantillonnage minimale d'environ 2 s.
Broker MQTT	Mosquitto ; reçoit les publications et les redistribue aux abonnés.
Bibliothèques ESP32	`WiFi`, `PubSubClient` (client MQTT), `DHT` (lecture capteur), `ArduinoJson` (v7, classe `JsonDocument`).
Node.js + Express 5	Serveur HTTP et distribution des fichiers statiques.
`mqtt` 5.x	Client MQTT côté serveur.
`socket.io` 4.x	Diffusion temps réel (WebSocket) vers les navigateurs.

Table 2 – Matériel et technologies utilisés.

2.3 Protocole MQTT et topics

MQTT est un protocole de messagerie léger fondé sur le modèle publication/abonnement : les émetteurs et les récepteurs ne se connaissent pas, ils passent uniquement par le broker, qui route les messages selon leur *topic*. Ce découplage permet d'ajouter un abonné (par exemple `mosquitto_sub`) sans modifier les ESP32.

Le projet utilise cinq topics sous le préfixe `smartroom/` (tableau 3).

Topic	Contenu	Publié
smartroom/measurements	JSON complet de la mesure (7 champs)	à chaque cycle
smartroom/temperature	même JSON complet	à chaque cycle
smartroom/humidity	même JSON complet	à chaque cycle
smartroom/status	même JSON complet	à chaque cycle
smartroom/alert	JSON d'alerte (3 champs)	si $T > 30\,°C$

Table 3 – Topics MQTT du projet.

Les quatre premiers topics transportent un message identique. Le format de la mesure est détaillé dans le tableau 4.

Champ	Type	Exemple	Signification
device_id	chaîne	`"ESP32_01"`	identifiant du nœud
temperature	nombre	`24`	température en °C (DHT22)
humidity	nombre	`70.9`	humidité relative en %
location	chaîne	`"Salle TP1"`	emplacement du nœud
sensor_type	chaîne	`"DHT22"`	type de capteur
counter	chaîne	`"1"`	valeur constante dans le code actuel
status	chaîne	`"OK"`	état declared par le nœud

Table 4 – Format JSON d'une mesure (environ 130 octets).

Le message d'alerte contient trois champs : `device_id`, `alert` (valeur `"HIGH_TEMPERATURE"`) et `temperature`.

Deux propriétés du transport sont à retenir. D'une part, la bibliothèque `PubSubClient` publie uniquement en QoS 0 : la livraison est « au plus une fois », et une mesure perdue n'est pas retransmise. D'autre part, aucun horodatage n'est inclus dans le message : l'ESP32 n'a pas d'horloge synchronisée dans ce code, et c'est le serveur qui ajoute la date de réception.

2.4 Cheminement d'une mesure

La figure 2 suit une mesure depuis le capteur jusqu'à l'écran, puis le cas particulier de l'alerte.

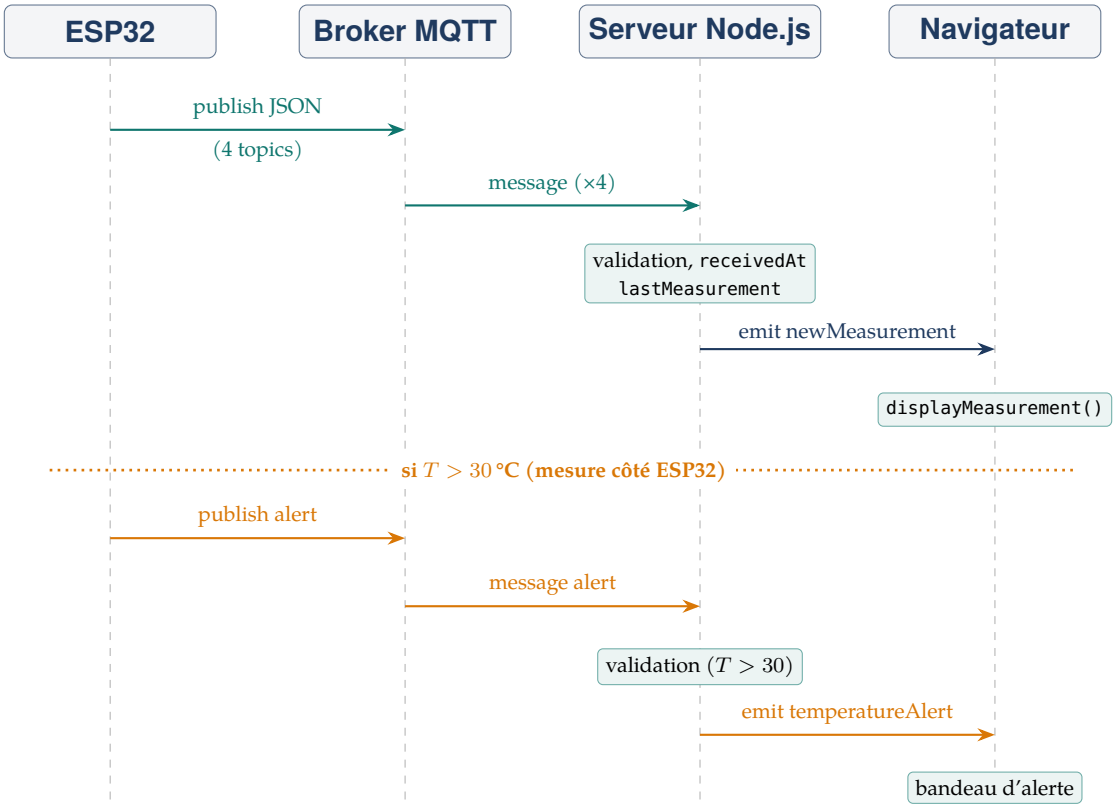


Figure 2 – Cheminement d'une mesure et d'une alerte.

3 Couche capteurs : firmware ESP32

3.1 Deux nœuds, un même programme

Les fichiers `main.c` et `esp2.c` contiennent le même programme Arduino (C++). Ils ne diffèrent que par trois lignes : la broche du capteur, l'identifiant de la mesure et l'emplacement (annexe **??**). Le tableau 5 compare les deux nœuds.

Paramètre	ESP32_01 (`main.c`)	ESP32_02 (`esp2.c`)	Remarque
Broche DHT22 (`DHTPIN`)	15	17	câblage distinct
`device_id` (mesures)	`ESP32_01`	`ESP32_02`	correct
`location`	`Salle TP1`	`Salle TP2`	correct
Identifiant client MQTT	`ESP32_01`	`ESP32_01`	**identique : voir §7.1**
`device_id` (alertes)	`ESP32_01`	`ESP32_01`	**identique : voir §7.1**

Table 5 – Comparaison des deux firmwares.

3.2 Connexion Wi-Fi et MQTT

Les identifiants Wi-Fi (`NOM_WIFI`, `MOT_DE_PASSE`) sont des valeurs à remplacer. La fonction `connectWiFi()` lance la connexion et attend, en affichant un point toutes les 500 ms. La fonction `reconnectMQTT()` boucle jusqu'à l'établissement de la session MQTT, avec une nouvelle tentative toutes les 3 s et l'affichage du code d'erreur `client.state()`.

```
29 void connectWiFi() {
30
31 WiFi.begin(ssid, password);
32
```

```
33 Serial.print("Connexion Wi-Fi");
34
35 while (WiFi.status() != WL_CONNECTED) {
36 delay(500);
37 Serial.print(".");
38 }
39
40 Serial.println("Wi-Fi connecte !");
41 Serial.println(WiFi.localIP());
42 }
43
44 void reconnectMQTT() {
45
46 while (!client.connected()) {
47
48 Serial.println("Connexion MQTT...");
49
50 if (client.connect("ESP32_01")) {
51
52 Serial.println("MQTT connecte !");
53
54 } else {
55
56 Serial.print("Erreur MQTT : ");
57 Serial.println(client.state());
58
59 delay(3000);
60 }
61 }
62 }
```

Code 1 – `main.c`, lignes 29–62 : connexion Wi-Fi et reconnexion MQTT

Ces deux fonctions sont rappelées au début de chaque tour de `loop()` si la connexion est perdue : le nœud se rétablit donc seul après une coupure du Wi-Fi ou du broker.

3.3 Cycle de mesure

La boucle principale (figure 3) n'utilise pas `delay(5000)` : elle compare `millis()` à l'instant de la dernière mesure. Cette approche non bloquante permet d'appeler `client.loop()` en continu, ce qui maintient la session MQTT active.

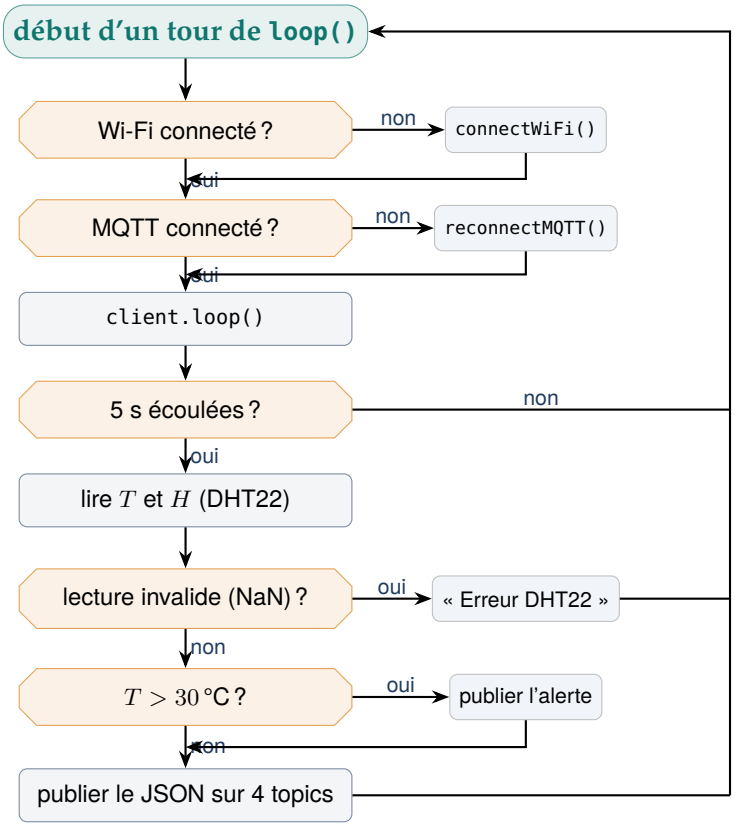


Figure 3 – Organigramme de `loop()`.

Après lecture, la valeur `NaN` signale un échec de lecture du DHT22. Le cycle est alors abandonné avec le message « Erreur DHT22 » et aucune donnée erronée n'est publiée.

```
87 if (millis() - previousMillis >= interval) {
88
89 previousMillis = millis();
90
91 float temperature = dht.readTemperature();
92 float humidity = dht.readHumidity();
93
94 if (isnan(temperature) || isnan(humidity)) {
95 Serial.println("Erreur DHT22");
96 return;
97 }
```

Code 2 – `main.c`, lignes 87–97 : cadencement par `millis()`, lecture et contrôle `NaN`

3.4 Détection de température élevée

Si la température dépasse 30,0 °C, le firmware publie un message d'alerte sur `smartroom/alert` *avant* la mesure normale. Tant que la température reste au-dessus du seuil, une alerte est publiée à chaque cycle (toutes les 5 s).

```
99 if (temperature > 30.0) {
100 JsonDocument alertDoc;
101 alertDoc["device_id"] = "ESP32_01";
102 alertDoc["alert"] = "HIGH_TEMPERATURE";
103 alertDoc["temperature"] = temperature;
104
105 String alertMessage;
```

```
serializeJson(alertDoc, alertMessage);

bool alertPublished = client.publish(topicAlert, alertMessage.c_str());

Serial.println(alertMessage);
Serial.println(
alertPublished ? "Alerte publiee" :
"Echec publication alerte"
);
}
```

Code 3 – `main.c`, lignes 99–115 : construction et publication de l'alerte

3.5 Construction et publication du message

La mesure est assemblée dans un `JsonDocument`, sérialisée dans une `String`, puis publiée à l'identique sur les quatre topics de mesure.

```
JsonDocument doc;

doc["device_id"] = "ESP32_01";
doc["temperature"] = temperature;
doc["humidity"] = humidity;
doc["location"] =  "Salle TP1";
doc["sensor_type"] =  "DHT22";
doc["counter"] =  "1";
doc["status"] =  "OK";

String message;

serializeJson(doc, message);

bool published = client.publish(
topicMesure,
message.c_str()
);

bool published1 = client.publish(
topicTemperature,
message.c_str()
);
bool published2 = client.publish(
topicHumidity,
message.c_str()
);
bool published3 = client.publish(
topicStatus,
message.c_str()
);
```

Code 4 – `main.c`, lignes 117–147 : message JSON et publication sur les quatre topics

4 Couche serveur : Node.js

4.1 Initialisation

Le serveur Express est enveloppé dans un serveur HTTP pour partager le port avec Socket.IO. Le dossier `public` est exposé en statique et `dashboard.html` sert de page d'accueil.

```
1  const express = require("express");
2  const http = require("http");
3  const mqtt = require("mqtt");
4  const { Server } = require("socket.io");
5
6  const app = express();
7  const server = http.createServer(app);
8  const io = new Server(server);
9
10 app.use(express.static("public", { index: "dashboard.html" }));
11
12 const mqttClient = mqtt.connect(
13 "mqtt://127.0.0.1:1883"
14 );
15
16 const topicMesure = "smartroom/measurements";
17 const topicTemperature = "smartroom/temperature";
18 const topicHumidity = "smartroom/humidity";
19 const topicStatus = "smartroom/status";
20 const topicAlert = "smartroom/alert";
21
22
23 let lastMeasurement = null;
```

Code 5 – `server.js`, lignes 1–23 : import des modules, serveur statique et connexion au broker

4.2 Abonnement MQTT et validation

Chaque message est converti de JSON en objet dans un bloc `try/catch`. Un `switch` sur le topic aiguille ensuite le traitement (tableau 6).

Topic reçu	Contrôles effectués	Événement Socket.IO
smartroom/measurements	`temperature` et `humidity` numériques ; `device_id` chaîne ; $T \in [-40\,;\,80]$; $H \in [0\,;\,100]$. Ajout de `receivedAt` et mémorisation dans `lastMeasurement`.	`newMeasurement`
smartroom/temperature	aucun (JSON relayé tel quel)	`newTemperature`
smartroom/humidity	aucun (JSON relayé tel quel)	`newHumidity`
smartroom/status	aucun (JSON relayé tel quel)	`deviceStatus`
smartroom/alert	`device_id` chaîne ; `alert` égal à `"HIGH_TEMPERATURE"` ; `temperature` numérique > 30. Ajout de `receivedAt`.	`temperatureAlert`

Table 6 – Traitement des messages côté serveur.

```
58             case topicMesure:
59
60                 // Complete measurement
61                 console.log("Mesure complete :", data);
62
63                 // Your existing validation
64                 if (
65                     typeof data.temperature !== "number" ||
66                     typeof data.humidity !== "number" ||
67                     typeof data.device_id !== "string" ||
```

```
                    data.temperature < -40 ||
                    data.temperature > 80 ||
                    data.humidity < 0 ||
                    data.humidity > 100
                ) {
                    console.log("Mesure invalide");
                    return;
                }

                const measurement = {
                    ...data,
                    receivedAt: new Date().toISOString()
                };

                lastMeasurement = measurement;

                io.emit("newMeasurement", measurement);

                break;
```

Code 6 – `server.js`, lignes 58–86 : validation et diffusion d'une mesure

```
            case topicAlert:

                if (
                    typeof data.device_id !== "string" ||
                    data.alert !== "HIGH_TEMPERATURE" ||
                    typeof data.temperature !== "number" ||
                    data.temperature <= 30
                ) {
                    console.log("Alerte invalide :", data);
                    return;
                }

                const alert = {
                    ...data,
                    receivedAt: new Date().toISOString()
                };

                console.log("ALERTE TEMPERATURE ELEVEE :", alert);
                io.emit("temperatureAlert", alert);

                break;
```

Code 7 – `server.js`, lignes 109–129 : validation et diffusion d'une alerte

4.3 API REST

La route `GET /api/measurements/latest` renvoie la dernière mesure valide (ou `null` avant la première réception), permettant l'affichage immédiat sur le tableau de bord.

```
app.get("/api/measurements/latest", (req, res) => {
res.json(lastMeasurement);
});

server.listen(3000, "0.0.0.0", () => {
console.log("Dashboard actif sur port 3000");
```

Code 8 – `server.js`, lignes 140–145 : route REST et démarrage du serveur

5 Interface : tableau de bord

5.1 Structure et logique cliente

La page `dashboard.html` comporte deux cartes (température et humidité), l'identifiant du dernier équipement, l'heure de réception, un indicateur de connexion et un paragraphe d'alerte.

```
42 <div class="dashboard">
43
44 <div class="card">
45 <h2>Temperature</h2>
46 <div class="value" id="temperature">--</div>
47 °C
48 </div>
49
50 <div class="card">
51 <h2>Humidite</h2>
52 <div class="value" id="humidity">--</div>
53 %
54 </div>
55
56 </div>
57
58 <p>Equipement : <span id="device">--</span></p>
59 <p>Derniere reception : <span id="time">--</span></p>
60 <p id="alert" hidden></p>
```

Code 9 – `dashboard.html`, lignes 42–60 : cartes de mesure et zones d'information

Le script ouvre la connexion Socket.IO, interroge l'API REST initiale, puis écoute en temps réel les événements transmis par le serveur.

```
157 function displayMeasurement(data) {
158
159 document.getElementById("temperature").textContent =
160 Number(data.temperature).toFixed(1);
161
162 document.getElementById("humidity").textContent =
163 Number(data.humidity).toFixed(1);
164
165 document.getElementById("device").textContent =
166 data.device_id;
167
168 document.getElementById("time").textContent =
169 new Date(data.receivedAt).toLocaleTimeString();
170 }
171
172 fetch("/api/measurements/latest")
173 .then(response => response.json())
174 .then(data => {
175 if (data) displayMeasurement(data);
176 });
177
178 socket.on("newMeasurement", displayMeasurement);
179
180 socket.on("temperatureAlert", data => {
181 const alert = document.getElementById("alert");
182 alert.hidden = false;
183 alert.textContent =
```

```
184  'ALERTE : temperature elevee (${Number(data.temperature).toFixed(1)} °C)';
185  });
```

Code 10 – `dashboard.html`, lignes 157–185 : mise à jour de l'affichage et gestion de l'alerte

6 Mise en œuvre et tests

6.1 Procédure de mise en œuvre

1. **Broker :** Démarrer Mosquitto sur `192.168.1.10:1883` en autorisant les connexions distantes (`listener 1883`, `allow_anonymous true`).
2. **ESP32 :** Téléverser `main.c` sur le premier nœud et `esp2.c` sur le second via l'IDE Arduino.
3. **Serveur :** Lancer `npm start` dans le dossier `nodeserver`.
4. **Tableau de bord :** Naviguer vers `http://<serveur-ip>:3000`.

6.2 Résultats observés

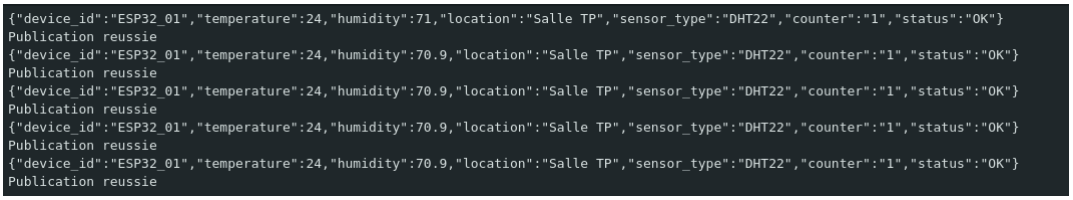


Figure 4 – Moniteur série de l'ESP32 : mesures publiées toutes les 5 secondes.

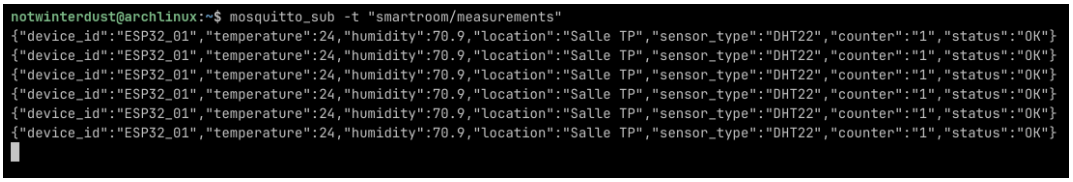


Figure 5 – Réception des mesures par `mosquitto_sub` sur le topic `smartroom/measurements`.

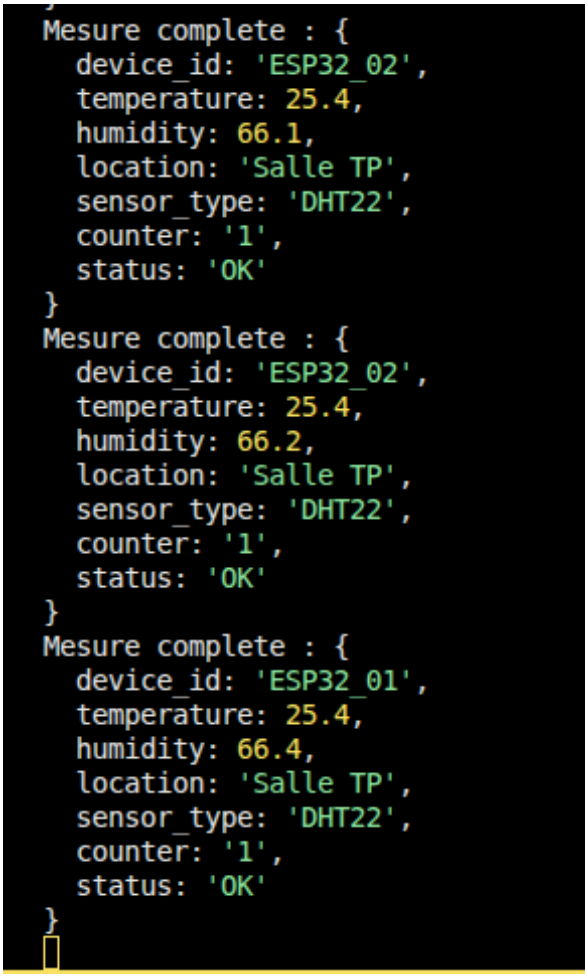


FIGURE 6 – Console du serveur Node.js : mesures reçues des deux ESP32.

6.3 Scénarios de test

Scénario	Résultat attendu	Statut
Mesure périodique	JSON publié toutes les 5 s sur 4 topics	Observé (fig. 4)
Abonné indépendant	`mosquitto_sub` reçoit les messages	Observé (fig. 5)
Deux nœuds, réception serveur	« Mesure complete » pour chaque `device_id`	Observé (fig. 6)
Température > 30 **°C**	alerte publiée, journalisée, affichée	À réaliser
Mesure hors plage / JSON invalide	« Mesure invalide », rien de diffusé	À réaliser
Coupure Wi-Fi / broker	reconnexion automatique du nœud	À réaliser

TABLE 7 – Scénarios de test et état de validation.

7 Analyse critique et pistes d'amélioration

7.1 Anomalies identifiées

N°	Fichier	Constat et conséquence	Correction proposée
1	dashboard.html	Marqueurs de conflit Git non résolus (lignes 71, 150 et 155). Erreur de syntaxe bloquante sur le script client.	Résoudre le conflit et harmoniser le script.
2	esp2.c	Identifiant client MQTT `"ESP32_01"` identique à `main.c`. Provoque des déconnexions en boucle.	Utiliser `client.connect("ESP32_02")`.
3	esp2.c	Le message d'alerte porte `device_id = "ESP32_01"`.	Corriger vers `"ESP32_02"`.
4	main.c / esp2.c	Le champ `counter` vaut toujours la chaîne `"1"`.	Entier incrémenté à chaque envoi.
5	package.json	Dépendance `socketio` obsolète/inutile.	Nettoyer avec `npm uninstall socketio`.

Table 8 – Anomalies relevées dans les scripts.

7.2 Pistes d'amélioration

— **Factorisation :** Regrouper les firmwares avec des constantes de configuration.

— **Hiérarchie des topics :** Adopter des topics dédiés par équipement (`smartroom/<device_id>/...`) avec des payloads optimisés.

— **Gestion des états :** Intégrer les messages « Last Will and Testament » (LWT) pour détecter la déconnexion inopinée d'un nœud.

— **Persistance :** Mémoriser l'historique dans une base de données (SQLite/InfluxDB) pour l'affichage de courbes temporelles.